Universidad de Antioquia
Curso Fundamentos Sistemas de Información
2026

Laboratorio Nro 2. Reglas del Negocio con Spring Boot y Drools

Introducción.

En este laboratorio se usará el framework Drools, el cual es un motor de reglas Java que se encarga de aplicar reglas de negocio dentro de nuestras aplicaciones. De esta manera aparece el concepto de Sistema de Administración de Reglas de Negocio (BRMS - Business Rules Management System), el cual permite parametrizar las reglas del negocio de forma externa a la aplicación que las utiliza.

Drools es un sistema de administración de reglas de negocio (BRMS) con un motor de reglas basado en una adaptación orientada a objetos del algoritmo Rete (https://es.wikipedia.org/wiki/Algoritmo_Rete). Permite expresar de una forma más natural las reglas de negocio interactuando con los objetos de negocio. Provee separación de lógica (reglas) y datos (hechos), por lo tanto se acerca al lenguaje común que necesita algún dominio del negocio.

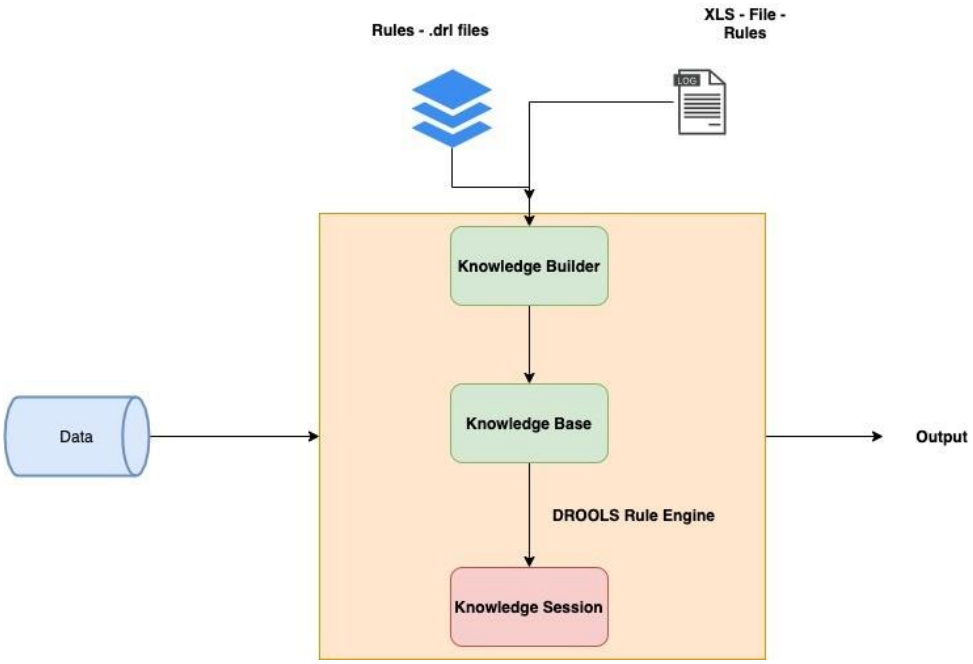
También provee soporte para la programación declarativa, y es lo suficientemente flexible para expresar la semántica del problema con un lenguaje específico de dominio (DSL). Drools cuenta con la implementación completa de la JSR-94 Rule Engine API (http://www.jcp.org/en/jsr/detail?id=94).

Para especificar las reglas Drools utiliza el lenguaje de reglas de drools (DRL) para especificar las condiciones, acciones y funciones de las mismas, las cuales se puede expresar con distintos lenguajes, como Java y MVEL. Entonces las reglas son guardadas en archivos de texto con la extensión drl. Esta plataforma es tan extensible que además puede definir las reglas del negocio en un archivo tipo Excel y enlazarlo hacia el motor de reglas.

Drools se puede utilizar como una parte de nuestro sistema, dónde se pueden dejar configurar por Drools las reglas de negocio que por ejemplo son muy cambiantes.

Arquitectura global de Drools.

En la siguiente figura se presenta la arquitectura general de Drools



Entre los principales elementos están:

- **Reglas:** son las condiciones del negocio.
- **Hechos:** representan los datos que sirven de entrada a las reglas.
- **Memoria de trabajo**: se precarga con las reglas configuradas y está preparada para la entrada de datos de coincidencia con los patrones.
- **Knowledge Builder:** El Knowledge Builder es responsable de tomar archivos fuente, como un archivo .drl, un archivo .bpmn2 o un archivo .xls, y convertirlos en un Paquete de Conocimiento de definiciones de reglas y procesos que una Base de Conocimiento puede consumir.
- **Base de conocimientos:** es el corazón del motor de reglas para tomar decisiones. Reúne la base de conocimientos que puede utilizarse para el razonamiento. Es la interfaz que gestiona una colección de reglas, procesos y tipos internos. La base de conocimiento proporciona métodos para crear sesiones de conocimiento.
- **Sesión de conocimiento**: se recupera de la base de conocimiento, se maneja para interactuar con el motor Drools. Drools soporta sesiones de conocimiento con y sin estado.
- **KIE (Knowledge Is Everything**) es el nuevo nombre que engloba drools, optaPlanner, jBPM, Guvnor, uberFire y otras tecnologías relacionadas. Para nuestros ejemplos, utilizaremos las API generadas por KIE (https://www.kiegroup.org/). El grupo KIE proporciona la implementación opensource de DROOLS.

Como definir las reglas del negocio.

Drools evalúa la aplicación de las reglas que definamos sobre los hechos que tenga en su memoria de trabajo.

Como vemos, en principio agrupamos las reglas en un paquete.

Luego necesitamos importar las clases java sobre las que operará el archivo de reglas.

Con **rule** definimos el nombre de la regla. Con la palabra reservada **when** se indica la condición de aplicación de la regla, es decir que aquí ponemos las condiciones que deben cumplir los objetos para aplicar la regla.

Con la coma separamos condiciones, que se evalúan como un Y lógico (conjunción).

Aquí también podemos poner disyunciones (O lógico) y negaciones.

En la sección del **then** especificamos las acciones que ejecutará la regla cuando se cumplan sus condiciones.

Si bien en la parte de las acciones se puede escribir cualquier código java, la idea es no usar condicionales. Una regla debe ser declarativa y atómica por naturaleza y luego Drools evalua cada regla a cada objeto que tenga en la memoria de trabajo, y la aplica a los que correspondan (que cumplan la condición).

Crear una aplicación spring boot con Drools.

Vamos a crear una aplicación Spring Boot con Drools para evaluar la asignación de créditos bancarios. La aplicación evaluará el perfil del cliente (ingresos, historial crediticio, edad) y el tipo de crédito solicitado (personal, hipotecario, automotriz) para determinar si se aprueba el crédito, el monto máximo y la tasa de interés.

Crear una aplicación spring boot con las dependencias necesarias a través de Spring Initializer (http://start.spring.io) . Añade la dependencia spring boot **starter web y lombok**. Además, incluya las dependencias de drools en el archivo de configuración pom XML de la aplicación spring boot.

NOTA: Para el proyecto y por temas de compatibilidad de las librerías usadas configuramos el proyecto usando la versión 3.2.5 de Spring Boot y la versión 17 del JDK de Java.

A continuación, se muestra el contenido completo del archivo **pom.xml.**

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-parent</artifactId>
       <version>3.2.5</version>
       <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.udea</groupId>
    <artifactId>droolrules</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>droolrules</name>
    <description>Demo project for Spring Boot</description>

    <properties>
       <java.version>17</java.version>
       <drools.version>7.74.1.Final</drools.version>
       <spring-ai.version>1.0.0-M6</spring-ai.version>
    </properties>
    <dependencies>

    <!--
       <dependency>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-starter-thymeleaf</artifactId>
       </dependency>-->




       <dependency>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-starter-validation</artifactId>
       </dependency>
       <dependency>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-starter-web</artifactId>
       </dependency>

       <dependency>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-devtools</artifactId>
          <scope>runtime</scope>
          <optional>true</optional>
       </dependency>
```

```
    <dependency>
       <groupId>org.drools</groupId>
       <artifactId>drools-core</artifactId>
       <version>${drools.version}</version>
    </dependency>

    <dependency>
       <groupId>org.drools</groupId>
       <artifactId>drools-compiler</artifactId>
       <version>${drools.version}</version>
    </dependency>

    <dependency>
       <groupId>org.kie</groupId>
       <artifactId>kie-api</artifactId>
       <version>${drools.version}</version>
    </dependency>

    <dependency>
       <groupId>org.drools</groupId>
       <artifactId>drools-decisiontables</artifactId>
       <version>${drools.version}</version>
    </dependency>

    <dependency>
       <groupId>org.drools</groupId>
       <artifactId>drools-mvel</artifactId>
       <version>${drools.version}</version>
    </dependency>


    <dependency>
       <groupId>org.springframework.ai</groupId>
       <artifactId>spring-ai-openai-spring-boot-starter</artifactId>
    </dependency>

    <dependency>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-test</artifactId>
       <scope>test</scope>
    </dependency>
 </dependencies>

 <repositories>
    <repository>
       <id>spring-milestones</id>
```

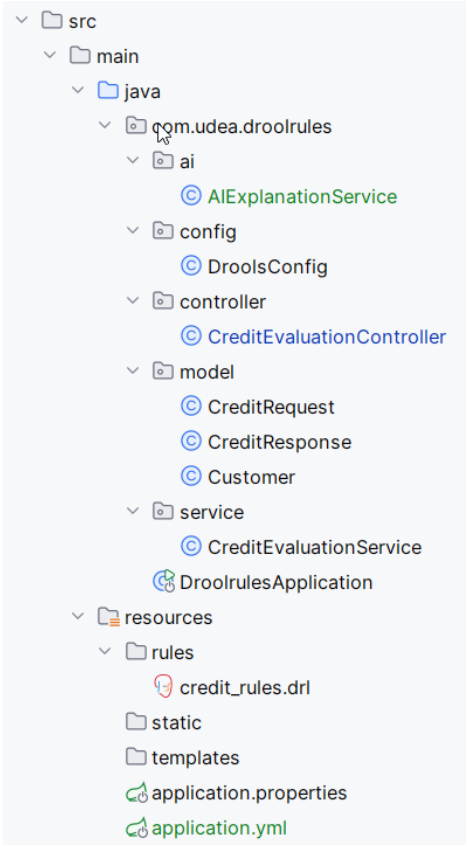
```
            <name>Spring Milestones</name>
            <url>https://repo.spring.io/milestone</url>
         </repository>
      </repositories>

      <dependencyManagement>
         <dependencies>
            <dependency>
               <groupId>org.springframework.ai</groupId>
               <artifactId>spring-ai-bom</artifactId>
               <version>${spring-ai.version}</version>
               <type>pom</type>
               <scope>import</scope>
            </dependency>
         </dependencies>
      </dependencyManagement>


      <build>
         <plugins>
            <plugin>
               <groupId>org.springframework.boot</groupId>
               <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
         </plugins>
      </build>

</project>
```

La estructura del proyecto será la siguiente:



Configurar la aplicación drools

Crea una clase java de configuración con el nombre **DroolsConfig**.

Añade el siguiente código a la clase java.

```
package com.udea.drools.config;


import org.kie.api.KieServices;
import org.kie.api.builder.KieBuilder;
import org.kie.api.builder.KieFileSystem;
import org.kie.api.builder.KieModule;
import org.kie.api.runtime.KieContainer;
import org.kie.internal.io.ResourceFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class DroolsConfig {

    private static final KieServices kieServices = KieServices.Factory.get();
    private static final String RULES_CUSTOMER_RULES_DRL = "rules/credit_rules.drl";

    @Bean
```

```
    public KieContainer kieContainer() {
        KieFileSystem kieFileSystem = kieServices.newKieFileSystem();

kieFileSystem.write(ResourceFactory.newClassPathResource(RULES_CUSTOMER_RULES_DRL));
        KieBuilder kb = kieServices.newKieBuilder(kieFileSystem);
        kb.buildAll();
        KieModule kieModule = kb.getKieModule();
        KieContainer kieContainer =
kieServices.newKieContainer(kieModule.getReleaseId());
        return kieContainer;
    }
}
```

La clase de configuración define un el bean **KieContainer** y se utiliza para construir el motor de reglas cargando los archivos de reglas bajo la carpeta **/resources** de la aplicación.

Creamos la instancia **KieFileSystem** y configuramos el motor de reglas y cargamos el archivo DRL desde el directorio resources de la aplicación.

También, podemos usar la instancia **KieBuilder** para construir el módulo drools. Podemos utilizar la instancia singleton **KieService** para crear la instancia KieBuilder.

Finalmente, usamos KieService para crear un KieContainer y configurarlo como un spring bean.

Añadir las clases del modelo

Cree una clase Pojo con el nombre **Customer** y defina los siguientes campos.

```
package com.udea.drools.model;

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

public class Customer {
    @NotBlank(message = "El nombre no puede estar vacío")
    @Size(min = 2, max = 100, message = "El nombre debe tener entre 2 y 100
caracteres")
    private String name;

    @NotNull(message = "La edad es obligatoria")
    @Min(value = 18, message = "La edad debe ser al menos 18 años")
    private int age;

    @NotNull(message = "Los ingresos mensuales son obligatorios")
    @Min(value = 0, message = "Los ingresos mensuales no pueden ser negativos")
    private double monthlyIncome;

    @NotNull(message = "El puntaje crediticio es obligatorio")
    @Min(value = 0, message = "El puntaje crediticio no puede ser negativo")
    @Min(value = 0, message = "El puntaje crediticio debe estar entre 0 y 1000")
    private int creditScore;

    // Constructor vacío
```

```
    public Customer() {}

    // Constructor con parámetros
    public Customer(String name, int age, double monthlyIncome, int creditScore) {
        this.name = name;
        this.age = age;
        this.monthlyIncome = monthlyIncome;
        this.creditScore = creditScore;
    }

    // Getters y Setters
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public double getMonthlyIncome() {
        return monthlyIncome;
    }

    public void setMonthlyIncome(double monthlyIncome) {
        this.monthlyIncome = monthlyIncome;
    }

    public int getCreditScore() {
        return creditScore;
    }

    public void setCreditScore(int creditScore) {
        this.creditScore = creditScore;
    }
}
```

Validaciones agregadas:

- name: No puede estar vacío (@NotBlank) y debe tener entre 2 y 100 caracteres (@Size).
- age: Obligatorio (@NotNull) y debe ser al menos 18 (@Min).
- monthlyIncome: Obligatorio (@NotNull) y no negativo (@Min).
- creditScore: Obligatorio (@NotNull) y no negativo (@Min).

Recibimos esta clase como un objeto de petición al motor de reglas y también enviamos estos campos como la entrada a las reglas definidas para calcular el importe de la tasa de interés para el pedido del cliente dado.

Ahora creamos la clase **CreditRequest** que tendrá el estatus de la solicitud y el valor de la tasa de interés.

```
package com.udea.drools.model;
```

```
import jakarta.validation.Valid;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;

public class CreditRequest {
    @NotNull(message = "El cliente es obligatorio")
    @Valid
    private Customer customer;

    @NotBlank(message = "El tipo de crédito es obligatorio")
    private String creditType; // personal, hipotecario, automotriz

    @NotNull(message = "El monto solicitado es obligatorio")
    @Min(value = 1000, message = "El monto solicitado debe ser al menos 1000")
    private double requestedAmount;

    // Constructor vacío
    public CreditRequest() {}

    // Constructor con parámetros
    public CreditRequest(Customer customer, String creditType, double
requestedAmount) {
        this.customer = customer;
        this.creditType = creditType;
        this.requestedAmount = requestedAmount;
    }

    // Getters y Setters
    public Customer getCustomer() {
        return customer;
    }

    public void setCustomer(Customer customer) {
        this.customer = customer;
    }

    public String getCreditType() {
        return creditType;
    }

    public void setCreditType(String creditType) {
        this.creditType = creditType;
    }

    public double getRequestedAmount() {
        return requestedAmount;
    }

    public void setRequestedAmount(double requestedAmount) {
```

```
        this.requestedAmount = requestedAmount;
    }
}
```

Ahora creamos la clase **CreditResponse** que usaremos para la respuesta a las diferentes solicitudes.

```
package com.udea.drools.model;

public class CreditResponse {
    private boolean approved;
    private double approvedAmount;
    private double interestRate;
    private String message;

    // Constructor vacío
    public CreditResponse() {}

    // Constructor con parámetros
    public CreditResponse(boolean approved, double approvedAmount, double
interestRate, String message) {
        this.approved = approved;
        this.approvedAmount = approvedAmount;
        this.interestRate = interestRate;
        this.message = message;
    }

    // Getters y Setters
    public boolean isApproved() {
        return approved;
    }

    public void setApproved(boolean approved) {
        this.approved = approved;
    }

    public double getApprovedAmount() {
        return approvedAmount;
    }

    public void setApprovedAmount(double approvedAmount) {
        this.approvedAmount = approvedAmount;
    }

    public double getInterestRate() {
        return interestRate;
    }

    public void setInterestRate(double interestRate) {
        this.interestRate = interestRate;
    }
```

```
    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }
}
```

Definir las reglas drools

Crea las drools rules dentro de un archivo con el nombre **credit_rules.drl**. Estas reglas definirán la lógica para aprobar o rechazar créditos según el perfil del cliente y el tipo de crédito

Añade el archivo
en el directorio **/src/main/resources/rules**.

```
package rules

import com.udea.drools.model.Customer
import com.udea.drools.model.CreditRequest
import com.udea.drools.model.CreditResponse

// Regla 1: Rechazar si el puntaje crediticio es menor a 500
rule "RejectLowCreditScore"
    when
        $request : CreditRequest(customer.creditScore < 500)
        $response : CreditResponse()
    then
        $response.setApproved(false);
        $response.setMessage("Solicitud rechazada: Puntaje crediticio demasiado bajo.");
end

// Regla 2: Rechazar si la edad es menor a 18
rule "RejectUnderAge"
    when
        $request : CreditRequest(customer.age < 18)
        $response : CreditResponse()
    then
        $response.setApproved(false);
        $response.setMessage("Solicitud rechazada: El cliente debe ser mayor de 18 años.");
end

// Regla 3: Crédito Personal - Evaluación
rule "PersonalCreditApproval"
    when
        $request : CreditRequest(creditType == "personal", customer.creditScore >= 500,
customer.monthlyIncome >= 1000)
        $response : CreditResponse()
    then
        $response.setApproved(true);
```

```
        $response.setApprovedAmount(Math.min($request.getRequestedAmount(),
$request.getCustomer().getMonthlyIncome() * 10));
        $response.setInterestRate($request.getCustomer().getCreditScore() > 700 ? 5.0 : 8.0);
        $response.setMessage("Crédito personal aprobado.");
end

// Regla 4: Crédito Hipotecario - Evaluación
rule "MortgageCreditApproval"
    when
        $request : CreditRequest(creditType == "hipotecario", customer.creditScore >= 600,
customer.monthlyIncome >= 3000)
        $response : CreditResponse()
    then
        $response.setApproved(true);
        $response.setApprovedAmount(Math.min($request.getRequestedAmount(),
$request.getCustomer().getMonthlyIncome() * 50));
        $response.setInterestRate($request.getCustomer().getCreditScore() > 750 ? 3.5 : 4.5);
        $response.setMessage("Crédito hipotecario aprobado.");
end

// Regla 5: Crédito Automotriz - Evaluación
rule "AutoCreditApproval"
    when
        $request : CreditRequest(creditType == "automotriz", customer.creditScore >= 550,
customer.monthlyIncome >= 1500)
        $response : CreditResponse()
    then
        $response.setApproved(true);
        $response.setApprovedAmount(Math.min($request.getRequestedAmount(),
$request.getCustomer().getMonthlyIncome() * 20));
        $response.setInterestRate($request.getCustomer().getCreditScore() > 700 ? 6.0 : 7.5);
        $response.setMessage("Crédito automotriz aprobado.");
end

// Regla 6: Rechazar si no cumple con requisitos mínimos por tipo de crédito
rule "RejectInvalidCreditType"
    when
        $request : CreditRequest()
        $response : CreditResponse(approved == false)
    then
        $response.setMessage($response.getMessage() != null ? $response.getMessage() :
"Solicitud rechazada: No cumple con los requisitos para el tipo de crédito solicitado.");
end
```

Aquí definimos las reglas de ejecución:

Regla 1: Rechaza si el puntaje crediticio es menor a 500.
Regla 2: Rechaza si el cliente es menor de 18 años.
Regla 3: Aprueba créditos personales si el puntaje es ≥500 y los ingresos ≥1000. El monto aprobado es hasta 10 veces los ingresos mensuales.
Regla 4: Aprueba créditos hipotecarios si el puntaje es ≥600 y los ingresos ≥3000. El monto aprobado es hasta 50 veces los ingresos mensuales.
Regla 5: Aprueba créditos automotrices si el puntaje es ≥550 y los ingresos ≥1500. El monto aprobado es hasta 20 veces los ingresos mensuales.
Regla 6: Asegura que las solicitudes no aprobadas tengan un mensaje por defecto.

Necesitamos importar los modelos que se están utilizando en el archivo DRL.

El archivo DRL puede contener una o varias reglas. Podemos utilizar la sintaxis mvel (http://mvel.documentnode.com/) para especificar las reglas. Además, cada regla puede describirse mediante la palabra clave **rule.**

Se puede utilizar la sintaxis **when-then** para definir las condiciones de una regla.

Basándonos en los valores de entrada de la solicitud de Pedido, añadiremos un descuento al resultado. Cada regla añade un descuento adicional a la variable de resultado global si la expresión de la regla coincide.

Añadir la capa de servicio
Crea una clase java de tipo **service** con el nombre **CreditEvaluationService**, para evaluar las solicitudes de crédito y añade el siguiente contenido.

```
package com.udea.drools.service;

import com.udea.drools.model.CreditRequest;
import com.udea.drools.model.CreditResponse;
import org.kie.api.runtime.KieContainer;
import org.kie.api.runtime.KieSession;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class CreditEvaluationService {

    @Autowired
    private KieContainer kieContainer;

    public CreditResponse evaluateCredit(CreditRequest request) {
        // Crear una respuesta inicial
        CreditResponse response = new CreditResponse();

        // Crear una sesión de Drools
        KieSession kieSession = kieContainer.newKieSession();

        try {
            // Insertar los hechos (request y response) en la sesión
            kieSession.insert(request);
            kieSession.insert(response);

            // Ejecutar todas las reglas
            kieSession.fireAllRules();
        } finally {
            // Liberar la sesión
            kieSession.dispose();
        }

        return response;
```

```
    }
}
```

Estamos inyectando la instancia **KieContainer** y creamos una instancia **KieSession.**

También estamos estableciendo un parámetro llamado CreditRequest que contendrá el resultado de la ejecución de las reglas.

Para pasar el objeto **request** al fichero DRL, podemos utilizar el método **insert().**

Luego ejecutamos todas las reglas llamando al método **fireAllRules().** y finalmente terminamos la sesión llamando al método **dispose() del KieSession**.

Añadir la API REST

Crear una clase API REST con el nombre **CreditEvaluationController** y añade el siguiente contenido.

```
package com.udea.droolrules.controller;

import com.udea.droolrules.ai.AIExplanationService;
import com.udea.droolrules.model.CreditRequest;
import com.udea.droolrules.model.CreditResponse;
import com.udea.droolrules.model.Customer;
import com.udea.droolrules.service.CreditEvaluationService;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
//import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;


@RestController
@RequestMapping("/credit")
public class CreditEvaluationController {

@Autowired
   private CreditEvaluationService evaluationService;
  @Autowired private AIExplanationService aiService;

//Endpoint REST
   @PostMapping("/api/evaluate")
   //@ResponseBody
   public CreditResponse evaluateCreditApi(@Valid @RequestBody CreditRequest request, BindingResult result) {
     // CreditResponse creditResponse = evaluationService.evaluateCredit(request);
     //  String explanation = aiService.explain(creditResponse.getMessage());
       if(result.hasErrors()){
          String errorMessage = result.getAllErrors().stream()
```

```
                .map(error-> error.getDefaultMessage())
                .reduce((msg1,msg2)-> msg1 + "; " + msg2)
                .orElse("Errores de validacion");
          return new CreditResponse(false,0.0,0.0,"Error de validacion: "+errorMessage);
       }
       return
         evaluationService.evaluateCredit(request);

    }

    @PostMapping("/evaluateCredit")
    public Object evaluate(@RequestBody CreditRequest customer){

       CreditResponse decision = evaluationService.evaluateCredit(customer);

       String explanation = aiService.explain(decision.getMessage());

       return new Object(){

          public final CreditResponse result = decision;
          public final String aiExplanation = explanation;

       };

    }


/*
    //Mostrar el formulario web
    @GetMapping("/form")
    public String showCreditForm(Model model) {
       CreditRequest creditRequest = new CreditRequest();
       creditRequest.setCustomer(new Customer());
       model.addAttribute("creditRequest", creditRequest);
       return "credit_form";
    }


//Procesar la solicitud desde el formulario web
    @PostMapping("/evaluate")
    public String evaluateCreditWeb(@Valid CreditRequest creditRequest, BindingResult result, Model model) {
       if(result.hasErrors()){
          return "credit_form";
       }
       CreditResponse creditResponse = evaluationService.evaluateCredit(creditRequest);
       model.addAttribute("creditResponse", creditResponse);
       return "credit_result";
    }*/
```

}

El controlador expone una API POST con el endpoint **/credit**. El endpoint espera un objeto **CustomerRequest** y devuelve la respuesta **CustomerResponse** calculada.

La anotación @Controller para manejar vistas HTML.
El endpoint REST se define como **/credit/api/evaluate** para diferenciarlo de las rutas web.
El método showCreditForm se usa para mostrar el formulario (/credit/form).
El método evaluateCreditWeb se usa para procesar el formulario y mostrar los resultados (/credit/evaluate).
Agregamos @Valid para validar el CreditRequest.
Usamos BindingResult para capturar errores de validación.
Si hay errores, devolvemos un CreditResponse con un mensaje que concatena todos los errores.

Capa de AI
Podremos ampliar la respuesta al usuario empleando el soporte de un modelo de IA por ejemplo con OpenAI a través de la definición de un prompt así

```
package com.udea.droolrules.ai;

import org.springframework.ai.chat.client.ChatClient;
import org.springframework.stereotype.Service;

@Service
public class AIExplanationService {

   private final ChatClient chatClient;

   public AIExplanationService(ChatClient.Builder builder) {
      this.chatClient = builder.build();
   }

   public String explain(String decision){

      String prompt = """
      Eres un experto en reglas bancarias.

      Explica el siguiente resultado generado por Drools.

      Resultado:
      %s

      Explica:
      - Qué regla se activó
      - Por qué se activó
      - Qué significa para el cliente
      """.formatted(decision);
```

```
    return chatClient.prompt()
        .user(prompt)
        .call()
        .content();

  }

}
```

Archivo application.yml

Debemos configurar los parámetros de conexión al API Key de Open AI (asegúrese de crear previamente un api key en la página principal de chatGPT). Usaremos el modelo gpt-4o-mini ya que es el que menos token consume.

```
spring:
 application:
  name: droolsrules

 ai:
  openai:
   api-key: sk-proj-<APIKEY>
   chat:
    options:
     model: gpt-4o-mini
```

Crear Vistas Web con Thymeleaf

Crearemos dos vistas:

Formulario de solicitud de crédito: Donde el usuario ingresará los datos del cliente y la solicitud de crédito.

Página de resultados: Mostrará el resultado de la evaluación (aprobado/rechazado, monto, tasa de interés, mensaje).

Crea un archivo HTML en **src/main/resources/templates/credit_form.html**

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
    <meta charset="UTF-8">
    <title>Solicitud de Crédito</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 40px;
        }
        .form-container {
            max-width: 500px;
            margin: auto;
```

```
        }
        label {
            display: block;
            margin: 10px 0 5px;
        }
        input, select {
            width: 100%;
            padding: 8px;
            margin-bottom: 5px;
        }
        .error {
            color: red;
            font-size: 0.9em;
            margin-bottom: 10px;
            display: block;
        }
        button {
            padding: 10px 20px;
            background-color: #4CAF50;
            color: white;
            border: none;
            cursor: pointer;
        }
        button:hover {
            background-color: #45a049;
        }
    </style>
</head>
<body>
<div class="form-container">
    <h2>Solicitud de Crédito</h2>
    <form th:action="@{/credit/evaluate}" th:object="${creditRequest}" method="post">
        <h3>Datos del Cliente</h3>
        <label for="name">Nombre:</label>
        <input type="text" id="name" th:field="${creditRequest.customer.name}">
        <span class="error" th:errors="${creditRequest.customer.name}"></span>

        <label for="age">Edad:</label>
        <input type="number" id="age" th:field="${creditRequest.customer.age}">
        <span class="error" th:errors="${creditRequest.customer.age}"></span>

        <label for="monthlyIncome">Ingresos Mensuales:</label>
        <input type="number" id="monthlyIncome"
th:field="${creditRequest.customer.monthlyIncome}" step="0.01">
        <span class="error" th:errors="${creditRequest.customer.monthlyIncome}"></span>

        <label for="creditScore">Puntaje Crediticio (0-1000):</label>
        <input type="number" id="creditScore"
th:field="${creditRequest.customer.creditScore}">
        <span class="error" th:errors="${creditRequest.customer.creditScore}"></span>

        <h3>Datos del Crédito</h3>
        <label for="creditType">Tipo de Crédito:</label>
        <select id="creditType" th:field="${creditRequest.creditType}">
            <option value="">Seleccione</option>
            <option value="personal">Personal</option>
            <option value="hipotecario">Hipotecario</option>
```

```
            <option value="automotriz">Automotriz</option>
        </select>
        <span class="error" th:errors="${creditRequest.creditType}"></span>

        <label for="requestedAmount">Monto Solicitado:</label>
        <input type="number" id="requestedAmount" th:field="${creditRequest.requestedAmount}"
step="0.01">
        <span class="error" th:errors="${creditRequest.requestedAmount}"></span>

        <button type="submit">Evaluar Solicitud</button>
    </form>
</div>
</body>
</html>
```

Notas:

- El formulario captura los datos del cliente (name, age, monthlyIncome, creditScore) y del crédito (creditType, requestedAmount).
- Usa Thymeleaf para vincular los campos al objeto CreditRequest (th:field).
- El formulario envía los datos a /credit/evaluate con el método POST.

Crea un archivo HTML en **src/main/resources/templates/credit_result.html**

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
  <meta charset="UTF-8">
  <title>Resultado de Evaluación</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 40px;
    }
    .result-container {
      max-width: 500px;
      margin: auto;
      padding: 20px;
      border: 1px solid #ccc;
      border-radius: 5px;
    }
    .approved {
      color: green;
    }
    .rejected {
      color: red;
    }
    a {
      display: inline-block;
      margin-top: 20px;
      text-decoration: none;
      color: #4CAF50;
```

```
    }
    a:hover {
      text-decoration: underline;
    }
  </style>
</head>
<body>
<div class="result-container">
  <h2>Resultado de la Evaluación</h2>
  <p>
    Estado:
    <span th:class="${creditResponse.approved} ? 'approved' : 'rejected'"
          th:text="${creditResponse.approved} ? 'Aprobado' : 'Rechazado'"></span>
  </p>
  <p th:if="${creditResponse.approved}">
    Monto Aprobado: <span th:text="'$' +
${#numbers.formatDecimal(creditResponse.approvedAmount, 0, 'COMMA', 2, 'POINT')}"></span>
  </p>
  <p th:if="${creditResponse.approved}">
    Tasa de Interés: <span th:text="${creditResponse.interestRate} + '%'"></span>
  </p>
  <p>Mensaje: <span th:text="${creditResponse.message}"></span></p>
  <a th:href="@{/credit/form}">Volver a solicitar</a>
</div>
</body>
</html>
```

Notas:

- Muestra el resultado de la evaluación (approved, approvedAmount, interestRate, message).
- Usa estilos condicionales para mostrar "Aprobado" en verde y "Rechazado" en rojo.
- Incluye un enlace para volver al formulario.

Probar la aplicación

Ejecuta la aplicación spring boot y accede al endpoint REST API enviando la petición JSON de pedido del cliente.

Para probar cada regla de Drools definida en `credit_rules.drl` de manera específica usaremos a Postman. Cada prueba estará diseñada para activar una regla concreta y verificar su comportamiento, asegurando que las demás reglas no interfieran (o que se active la regla de rechazo genérico cuando corresponda). Las pruebas incluyen el método URL, encabezados, cuerpo JSON, y una descripción del resultado esperado. Verifique que la aplicación está corriendo en **http://localhost:8080** y que el endpoint REST es **/credit/api/evaluate**.

Pruebas Individuales en Postman para las Reglas de Drools

Configuración General para Todas las Pruebas

- Método: POST
- URL: `http://localhost:8080/credit/api/evaluate`
- Encabezado:
  - Key: `Content-Type`
  - Value: `application/json`

A continuación, las pruebas individuales para cada regla:

Universidad de Antioquia
Curso Fundamentos Sistemas de Información
2026

Prueba 1: Regla "RejectLowCreditScore"
-Nombre: `Test RejectLowCreditScore`
- Descripción: Prueba que la solicitud se rechace cuando el puntaje crediticio es menor a 500, independientemente del tipo de crédito.
- Cuerpo de la Solicitud (Body -> Raw -> JSON):

```
{
      "customer": {
          "name": "Ana Gómez",
          "age": 25,
          "monthlyIncome": 2000.0,
          "creditScore": 450
      },
      "creditType": "personal",
      "requestAmount": 10000.0
}
```

- Resultado Esperado:
- Status: 200 OK
- Respuesta:

```
{
    "approved": false,
    "approvedAmount": 0.0,
    "interestRate": 0.0,
    "message": "Solicitud rechazada: Puntaje crediticio demasiado bajo."
}
```

Notas: El puntaje de 450 es menor a 500, activando esta regla antes de que se evalúen las reglas específicas del tipo de crédito.

Prueba 2: Regla "RejectUnderAge"
- Nombre: Test RejectUnderAge
- Descripción: Prueba que la solicitud se rechace por edad menor a 18, incluso si el puntaje crediticio y los ingresos son suficientes. (Nota: La validación de Spring Validation también rechazará esto antes de Drools, pero ajustaremos el caso para que pase la validación y llegue a Drools si fuera necesario).
- Cuerpo de la Solicitud

```
{
      "customer": {
          "name": "Luis Martínez",
          "age": 16,
          "monthlyIncome": 2000.0,
          "creditScore": 700
      },
      "creditType": "personal",
      "requestAmount": 10000.0
}
```

- Resultado Esperado:

- Status: 200 OK
- Respuesta:

```
{
       "approved": false,
       "approvedAmount": 0.0,
       "interestRate": 0.0,
       "message": "Error de validación: La edad debe ser al menos 18 años"
   }
```

- Notas: En este caso, Spring Validation (`@Min(value = 18)`) intercepta la solicitud antes de que Drools evalúe la regla `RejectUnderAge`. Si desea probar específicamente la regla de Drools, tendrá que desactivar temporalmente la validación en `Customer.java` (quitar `@Min(value = 18)`), y el mensaje sería:

```
{
     "approved": false,
     "approvedAmount": 0.0,
     "interestRate": 0.0,
     "message": "Solicitud rechazada: El cliente debe ser mayor de 18 años."
}
```

Prueba 3: Regla "PersonalCreditApproval" - Tasa Baja
- Nombre: `Test PersonalCreditApproval Low Rate`
- Descripción: Prueba la aprobación de un crédito personal con puntaje > 700 (tasa 5.0%) y monto dentro del límite.
- Cuerpo de la Solicitud:

```
{
    "customer": {
        "name": "Juan Pérez",
        "age": 30,
        "monthlyIncome": 2000.0,
        "creditScore": 750
    },
    "creditType": "personal",
    "requestAmount": 15000.0
}
```

- Resultado Esperado:
- Status: 200 OK
- Respuesta:

```
   {
       "approved": true,
       "approvedAmount": 15000.0,
       "interestRate": 5.0,
       "message": "Crédito personal aprobado."
   }
```

- Notas: Cumple los requisitos (`creditScore >= 500`, `monthlyIncome >= 1000`), y el puntaje > 700 asegura la tasa baja. El monto solicitado (15,000) es menor al máximo permitido (20,000 = 10x ingresos).

Prueba 4: Regla "PersonalCreditApproval" - Tasa Alta
- Nombre: `Test PersonalCreditApproval High Rate`

- Descripción: Prueba la aprobación de un crédito personal con puntaje entre 500 y 700 (tasa 8.0%) y monto limitado por ingresos.

- Cuerpo de la Solicitud:

```
{
    "customer": {
        "name": "María López",
        "age": 28,
        "monthlyIncome": 1500.0,
        "creditScore": 600
    },
    "creditType": "personal",
    "requestAmount": 20000.0
}
```

- Resultado Esperado:
- Status: 200 OK
- Respuesta:

```
{
    "approved": true,
    "approvedAmount": 15000.0,
    "interestRate": 8.0,
    "message": "Crédito personal aprobado."
}
```

- Notas: El monto solicitado (20,000) excede el máximo permitido (15,000 = 10x ingresos), por lo que se ajusta a 15,000. El puntaje de 600 (< 700) da una tasa de 8.0%.

Prueba 5: Regla "MortgageCreditApproval" - Tasa Baja
- Nombre: `Test MortgageCreditApproval Low Rate`
- Descripción: Prueba la aprobación de un crédito hipotecario con puntaje > 750 (tasa 3.5%).
- Cuerpo de la Solicitud:

```
{
    "customer": {
        "name": "Carlos García",
        "age": 35,
        "monthlyIncome": 4000.0,
        "creditScore": 800
    },
    "creditType": "hipotecario",
    "requestAmount": 150000.0
}
```

- Resultado Esperado:
- Status: 200 OK
- Respuesta:

```
{
    "approved": true,
    "approvedAmount": 150000.0,
    "interestRate": 3.5,
    "message": "Crédito hipotecario aprobado."
}
```


- Notas: Cumple los requisitos (`creditScore >= 600`, `monthlyIncome >= 3000`), y el puntaje > 750 asegura la tasa baja. El monto solicitado (150,000) es menor al máximo permitido (200,000 = 50x ingresos).

Prueba 6: Regla "MortgageCreditApproval" - Tasa Alta
- Nombre: `Test MortgageCreditApproval High Rate`
- Descripción: Prueba la aprobación de un crédito hipotecario con puntaje entre 600 y 750 (tasa 4.5%) y monto limitado por ingresos.
- Cuerpo de la Solicitud:


```
{
    "customer": {
        "name": "Sofía Ramírez",
        "age": 40,
        "monthlyIncome": 3500.0,
        "creditScore": 650
    },
    "creditType": "hipotecario",
    "requestAmount": 200000.0
}
```


- Resultado Esperado:
 - Status: 200 OK
 - Respuesta:


```
{
    "approved": true,
    "approvedAmount": 175000.0,
    "interestRate": 4.5,
    "message": "Crédito hipotecario aprobado."
}
```


- Notas: El monto solicitado (200,000) excede el máximo permitido (175,000 = 50x ingresos), por lo que se ajusta a 175,000. El puntaje de 650 (< 750) da una tasa de 4.5%.

Prueba 7: Regla "AutoCreditApproval" - Tasa Baja
-Nombre: `Test AutoCreditApproval Low Rate`
-Descripción: Prueba la aprobación de un crédito automotriz con puntaje > 700 (tasa 6.0%).
- Cuerpo de la Solicitud:


```
{
    "customer": {
        "name": "Pedro Sánchez",
```

```
            "age": 32,
            "monthlyIncome": 2500.0,
            "creditScore": 720
        },
        "creditType": "automotriz",
        "requestAmount": 30000.0
    }
```

- Resultado Esperado:
 - Status: 200 OK
 - Respuesta:

```
    {
        "approved": true,
        "approvedAmount": 30000.0,
        "interestRate": 6.0,
        "message": "Crédito automotriz aprobado."
    }
```

- Notas: Cumple los requisitos (`creditScore >= 550`, `monthlyIncome >= 1500`), y el puntaje > 700 asegura la tasa baja. El monto solicitado (30,000) es menor al máximo permitido (50,000 = 20x ingresos).

Prueba 8: Regla "AutoCreditApproval" - Tasa Alta
- Nombre: `Test AutoCreditApproval High Rate`
- Descripción: Prueba la aprobación de un crédito automotriz con puntaje entre 550 y 700 (tasa 7.5%) y monto limitado por ingresos.
- Cuerpo de la Solicitud:

```
    {
        "customer": {
            "name": "Laura Torres",
            "age": 29,
            "monthlyIncome": 1800.0,
            "creditScore": 600
        },
        "creditType": "automotriz",
        "requestAmount": 40000.0
    }
```

- Resultado Esperado:
 - Status: 200 OK
 - Respuesta:

```
    {
        "approved": true,
        "approvedAmount": 36000.0,
        "interestRate": 7.5,
        "message": "Crédito automotriz aprobado."
    }
```

- Notas: El monto solicitado (40,000) excede el máximo permitido (36,000 = 20x ingresos), por lo que se ajusta a 36,000. El puntaje de 600 (< 700) da una tasa de 7.5%.

Prueba 9: Regla "RejectInvalidCreditType"
- Nombre: `Test RejectInvalidCreditType`
- Descripción: Prueba el rechazo cuando no se cumplen los requisitos mínimos para el tipo de crédito solicitado (por ejemplo, ingresos insuficientes para un crédito hipotecario).
- Cuerpo de la Solicitud:

```
{
    "customer": {
        "name": "Diego Morales",
        "age": 27,
        "monthlyIncome": 2000.0,
        "creditScore": 650
    },
    "creditType": "hipotecario",
    "requestAmount": 100000.0
}
```

- Resultado Esperado:
 - Status: 200 OK
 - Respuesta:

```
{
       "approved": false,
       "approvedAmount": 0.0,
       "interestRate": 0.0,
       "message": "Solicitud rechazada: No cumple con los requisitos para el tipo de
crédito solicitado."
    }
```

- Notas: Los ingresos (2000) son menores a 3000, por lo que no se aprueba el crédito hipotecario, y esta regla genérica se activa después de que las reglas específicas fallen.

Cómo Configurar y Ejecutar Cada Prueba en Postman

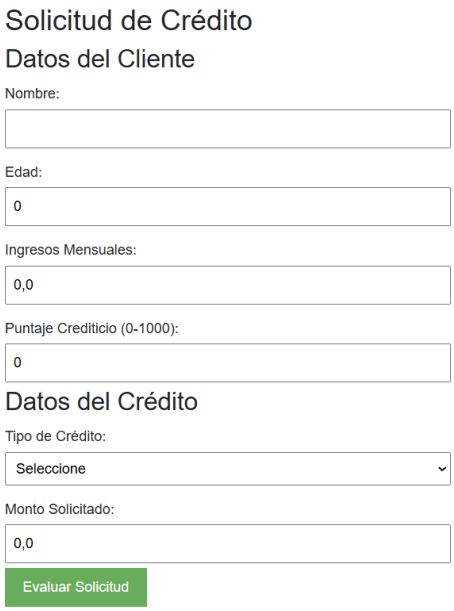
Configure Cada Prueba Individualmente:
 - Abre Postman.
 - Cree una nueva solicitud (haz clic en New -> HTTP Request).
 - Configure el método (POST), URL (`http://localhost:8080/credit/api/evaluate`), y encabezado (`Content-Type: application/json`).
 - Vaya a la pestaña Body, seleccione Raw, elige JSON, y pegue el cuerpo de la solicitud correspondiente (por ejemplo, el JSON de `Test RejectLowCreditScore`).
 - Dale un nombre a la solicitud (por ejemplo, `Test RejectLowCreditScore`) y guárdala en una colección (opcional).
Ejecute la Prueba:

- Haz clic en Send.
- Verifique la respuesta en la sección Body de Postman contra el "Resultado Esperado" proporcionado.

Notas Importantes

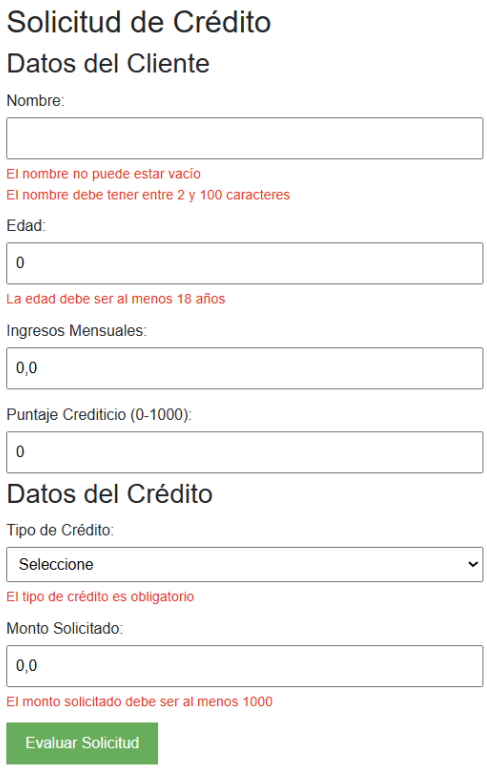
- **Orden de Ejecución:** Las reglas en `**credit_rules.drl**` tienen una prioridad implícita basada en su orden (las primeras reglas como `RejectLowCreditScore` se evalúan antes). Esto asegura que rechazos generales (puntaje bajo, edad) se activen antes de las reglas específicas de aprobación.
- **Validaciones de Spring:** Algunas reglas (como `RejectUnderAge`) pueden ser interceptadas por las validaciones de Spring Validation antes de llegar a **Drools**. Si quiere probar exclusivamente las reglas de Drools, puede desactivar temporalmente las anotaciones de validación en `Customer.java` y `CreditRequest.java` (por ejemplo, quitar `@Min(value = 18)` o `@NotBlank`), pero esto no es necesario para el comportamiento estándar de la aplicación.

Abre un navegador y ve a **http://localhost:8080/credit/form**. Verás un formulario donde puedes ingresar los datos del cliente y la solicitud de crédito.

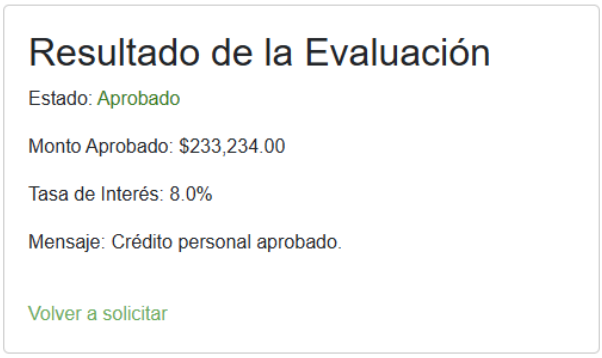


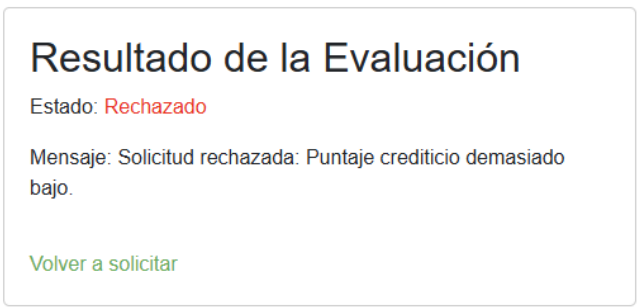
Puede probar las validaciones en la entrada de datos

Universidad de Antioquia
Curso Fundamentos Sistemas de Información
2026



De acuerdo con los datos ingresados puede observar el resultado de la evaluación crediticia según sea el caso:





Repositorio: https://github.com/diegobotia/lab2fsi2026

Universidad de Antioquia
Curso Fundamentos Sistemas de Información
2026

Ejercicio: Fecha de entrega 08 de Octubre de 2026

Cree una aplicación para crear un conjunto de 10 reglas en Drools (.drl) para una aerolínea.
Las reglas propuestas deben cumplir lo siguiente:

Explicación de las Reglas

1. **UpgradeToBusinessClassForFrequentFlyersWithDelays**:
   - Condición: Pasajero con estatus "Gold" o "Platinum" y un vuelo con más de 60 minutos de retraso.
   - Acción: Ascenso a clase ejecutiva.
2. **PriorityCheckInForSeniors**:
   - Condición: Pasajero mayor de 65 años.
   - Acción: Activar check-in prioritario.
3. **DiscountForLightLuggage**:
   - Condición: Equipaje menor a 10 kg.
   - Acción: Aplicar un descuento del 10%.
4. **DenyUpgradeForOverweightLuggage**:
   - Condición: Equipaje mayor a 23 kg.
   - Acción: Marcar al pasajero como no elegible para ascensos.
5. **AssignEmergencyExitSeatToYoungAdults**:
   - Condición: Pasajero entre 18 y 40 años con preferencia de asiento "Any" y un asiento de salida de emergencia disponible.
   - Acción: Asignar el asiento de salida de emergencia y marcarlo como ocupado.
6. **CompensationForExtremeDelays**:
   - Condición: Vuelo con retraso mayor a 180 minutos (3 horas).
   - Acción: Otorga una compensación de $200 al pasajero.
7. **ExtraLoyaltyPointsForLongFlights**:
   - Condición: Pasajero con membresía distinta a "Basic" en un vuelo de más de 5 horas.
   - Acción: Agrega 500 puntos de lealtad.
8. **RestrictLuggageOnShortFlights**:
   - Condición: Equipaje mayor a 15 kg en un vuelo de menos de 2 horas.
   - Acción: Marca el equipaje como no permitido.
9. **VipLoungeAccessForPlatinumMembers**:
   - Condición: Pasajero con membresía "Platinum".
   - Acción: Otorga acceso al salón VIP.
10. **PreferentialSeatForFamilies**:
    - Condición: Pasajero viajando con niños y sin preferencia específica de asiento.
    - Acción: Asigna un asiento preferencial para familias.

Entregar informe y código fuente .

Recursos Bibliográficos recomendados

1. Drools y Spring Boot. https://www.paradigmadigital.com/dev/drools-spring-boot-introduccion/
2. Página official de Drools. https://www.drools.org/
3. Lenguaje DRL. https://docs.jboss.org/drools/release/5.2.0.Final/drools-expert-docs/html/ch05.html
4. Tutorial Drools. https://hobsoft.com/guides/drools-tutorial-drools-rule-engine-architecture-examples/
5. QuickStart with Spring Boot Drools Rule Engine. https://medium.com/@aalam-info-solutions-llp/quickstart-with-spring-boot-drools-rule-engine-408477bac0b3

6. Tutorial de Motor de Reglas Drools. https://hkrtrainings.com/drools-tutorial
7. Drools Spring Integration. https://www.baeldung.com/drools-spring-integration
8. Spring Boot and Drools Example. https://www.techgeeknext.com/spring-boot/drools-example
9. Drools Tutorial: Drools Rule Engine Architecture & Examples. https://www.guru99.com/drools-tutorial.html